JobSheet - Week 4 - Polymorphism

Nama : **Raza Mulki Firdaus**

NIM : **251524120**

Kelas : **1D**

Repo GitHub : **https://github.com/zaaraza/Teknik-Pemrograman-**

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W4_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W3_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Inheritance via Polymorphism (Another Type of Employee)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 1-Polymorphism via Inheritance.pdf
Source Code
Commision.java

```java

//*******************************************************************
// Commission.java
//
// Represents an hourly employee who also earns commission on sales.
//*******************************************************************

public class Commission extends Hourly {
    private double totalSales; // total sales made by this employee
    private double commissionRate; // commission rate (decimal form, e.g. 0.20 = 20%)

    // -----------------------------------------------------------------
    // Sets up a commission employee using the specified information.
    // Parameters: name, address, phone, social security number,
    // hourly pay rate, commission rate
    // -----------------------------------------------------------------
    public Commission(String eName, String eAddress, String ePhone,
            String socSecNumber, double rate, double commRate) {
        super(eName, eAddress, ePhone, socSecNumber, rate);
        commissionRate = commRate;
        totalSales = 0;
    }

    // -----------------------------------------------------------------
    // Adds the specified sales amount to this employee's total sales.
    // -----------------------------------------------------------------
    public void addSales(double sales) {
        totalSales += sales;
    }

    // -----------------------------------------------------------------
    // Computes and returns the pay for this commission employee,
    // which is the hourly pay (from parent) plus commission on sales.
    // Resets totalSales to 0 after payment.
    // -----------------------------------------------------------------
    public double pay() {
        double payment = super.pay() + (totalSales * commissionRate);
        totalSales = 0;
        return payment;
    }

    // -----------------------------------------------------------------
    // Returns information about this commission employee as a string.
    // -----------------------------------------------------------------
    public String toString() {
        String result = super.toString();
        result += "\nTotal Sales: " + totalSales;
        return result;
    }
}
```

Employee.java

```java
//********************************************************************
// Employee.java Author: Lewis/Loftus
//
// Represents a general paid employee.
//********************************************************************
public class Employee extends StaffMember{
    protected String socialSecurityNumber;
    protected double payRate;
//-----------------------------------------------------------------
// Sets up an employee with the specified information.
//-----------------------------------------------------------------
    public Employee (String eName, String eAddress, String ePhone,String socSecNumber, double rate){
        super (eName, eAddress, ePhone);
        socialSecurityNumber = socSecNumber;
        payRate = rate;
    }
//-----------------------------------------------------------------
// Returns information about an employee as a string.
//-----------------------------------------------------------------
    public String toString(){
        String result = super.toString();
        result += "\nSocial Security Number: " + socialSecurityNumber;
        return result;
    }
//-----------------------------------------------------------------
// Returns the pay rate for this employee.
//-----------------------------------------------------------------
    public double pay(){
    return payRate;
    }
}
```

Executive.java

```java
//*******************************************************************
// Executive.java Author: Lewis/Loftus
//
// Represents an executive staff member, who can earn a bonus.
//*******************************************************************
public class Executive extends Employee{
    private double bonus;
    //-----------------------------------------------------------------
    // Sets up an executive with the specified information.
    //-----------------------------------------------------------------
    public Executive (String eName, String eAddress, String ePhone,
    String socSecNumber, double rate){
        super (eName, eAddress, ePhone, socSecNumber, rate);
        bonus = 0; // bonus has yet to be awarded
    }
    //-----------------------------------------------------------------
    // Awards the specified bonus to this executive.
    //-----------------------------------------------------------------
    public void awardBonus (double execBonus){
        bonus = execBonus;
    }
    //-----------------------------------------------------------------
    // Computes and returns the pay for an executive, which is the
    // regular employee payment plus a one-time bonus.
    //-----------------------------------------------------------------
    public double pay(){
        double payment = super.pay() + bonus;
        bonus = 0;
        return payment;
    }
}
```

Firm.java (main program)

```java
//*******************************************************************
// Firm.java Author: Lewis/Loftus
//
// Demonstrates polymorphism via inheritance.
//*******************************************************************
public class Firm {
    // ----------------------------------------------------------------
    // Creates a staff of employees for a firm and pays them.
    // ----------------------------------------------------------------
    public static void main(String[] args) {
        Staff personnel = new Staff();
        personnel.payday();
    }
}
```

Hourly.java

```java
//*******************************************************************
// Hourly.java Author: Lewis/Loftus
//
// Represents an employee that gets paid by the hour.
//*******************************************************************
public class Hourly extends Employee{
    private int hoursWorked;
    //-----------------------------------------------------------------
    // Sets up this hourly employee using the specified information.
    //-----------------------------------------------------------------
    public Hourly (String eName, String eAddress, String ePhone,
    String socSecNumber, double rate){
        super (eName, eAddress, ePhone, socSecNumber, rate);
        hoursWorked = 0;
    }
    //-----------------------------------------------------------------
    // Adds the specified number of hours to this employee's
    // accumulated hours.
    //-----------------------------------------------------------------
    public void addHours (int moreHours){
        hoursWorked += moreHours;
    }
    //-----------------------------------------------------------------
    // Computes and returns the pay for this hourly employee.
    //-----------------------------------------------------------------
    public double pay(){
        double payment = payRate * hoursWorked;
        hoursWorked = 0;
        return payment;
    }
    //-----------------------------------------------------------------
    // Returns information about this hourly employee as a string.
    //-----------------------------------------------------------------
    public String toString(){
        String result = super.toString();
        result += "\nCurrent hours: " + hoursWorked;
        return result;
    }
}
```

Staff.java

```java
//********************************************************************
// Staff.java Author: Lewis/Loftus (modified)
//
// Represents the personnel staff of a particular business.
//********************************************************************
public class Staff {
        StaffMember[] staffList;

    // -----------------------------------------------------------------
    // Sets up the list of staff members.
    // -----------------------------------------------------------------
public Staff() {
        staffList = new StaffMember[8]; // increased from 6 to 8

        staffList[0] = new Executive("Raza", "123 Main Line",
                "555-0469", "123-45-6789", 2423.07);
        staffList[1] = new Employee("Garin", "456 Off Line",
                "555-0101", "987-65-4321", 1246.15);
        staffList[2] = new Employee("Nafis", "789 Off Rocker",
                "555-0000", "010-20-3040", 1169.23);
        staffList[3] = new Hourly("Akmal", "678 Fifth Ave.",
                "555-0690", "958-47-3625", 10.55);
        staffList[4] = new Volunteer("Julfi", "987 Suds Blvd.",
                "555-8374");
        staffList[5] = new Volunteer("Uda Irham", "321 Duds Lane",
                "555-7282");

        // Two new Commission employees
        staffList[6] = new Commission("Ziddan", "100 Commerce St.",
                "555-1111", "111-22-3333", 6.25, 0.20);
        staffList[7] = new Commission("Lexy", "200 Market Ave.",
                "555-2222", "444-55-6666", 9.75, 0.15);

        ((Executive) staffList[0]).awardBonus(500.00);
        ((Hourly) staffList[3]).addHours(40);

        // Set up hours and sales for commission employees
        ((Commission) staffList[6]).addHours(35);
        ((Commission) staffList[6]).addSales(400);

        ((Commission) staffList[7]).addHours(40);
        ((Commission) staffList[7]).addSales(950);
    }

    // -----------------------------------------------------------------
    // Pays all staff members.
    // -----------------------------------------------------------------
    public void payday() {
        double amount;
        for (int count = 0; count < staffList.length; count++) {
            System.out.println(staffList[count]);
            amount = staffList[count].pay(); // polymorphic
            if (amount == 0.0)
                System.out.println("Thanks!");
            else
                System.out.println("Paid: " + amount);
            System.out.println("-----------------------------------");
        }
    }
}
```

StaffMember.java

```java
//*******************************************************************
// StaffMember.java Author: Lewis/Loftus
//
// Represents a generic staff member.
//*******************************************************************
abstract public class StaffMember{
    protected String name;
    protected String address;
    protected String phone;
//-----------------------------------------------------------------
// Sets up a staff member using the specified information.
//-----------------------------------------------------------------
public StaffMember (String eName, String eAddress, String ePhone){
    name = eName;
    address = eAddress;
    phone = ePhone;
    }
//-----------------------------------------------------------------
// Returns a string including the basic employee information.
//-----------------------------------------------------------------
public String toString(){
    String result = "Name: " + name + "\n";
    result += "Address: " + address + "\n";
    result += "Phone: " + phone;
    return result;
    }
//-----------------------------------------------------------------
// Derived classes must define the pay method for each type of
// employee.
//-----------------------------------------------------------------
    public abstract double pay();
}
```

Volunteer.java

```java
//*******************************************************************
// Volunteer.java Author: Lewis/Loftus
//
// Represents a staff member that works as a volunteer.
//*******************************************************************
public class Volunteer extends StaffMember{
//-----------------------------------------------------------------
// Sets up a volunteer using the specified information.
//-----------------------------------------------------------------
    public Volunteer (String eName, String eAddress, String ePhone)
    {
    super (eName, eAddress, ePhone);
    }
//-----------------------------------------------------------------
// Returns a zero pay value for this volunteer.
//-----------------------------------------------------------------
public double pay(){
    return 0.0;
    }
}
```

Screenshot Hasil

```
Name: Raza
Address: 123 Main Line
Phone: 555-0469
Social Security Number: 123-45-6789
Paid: 2923.07
-----------------------------------
Name: Garin
Address: 456 Off Line
Phone: 555-0101
Social Security Number: 987-65-4321
Paid: 1246.15
-----------------------------------
Name: Nafis
Address: 789 Off Rocker
Phone: 555-0000
Social Security Number: 010-20-3040
Paid: 1169.23
-----------------------------------
Name: Akmal
Address: 678 Fifth Ave.
Phone: 555-0690
Social Security Number: 958-47-3625
```

Penjelasan Permasalahan dan Solusi

Soal ini meminta pembuatan class Commission yang merupakan turunan dari class Hourly. Class tersebut harus merepresentasikan karyawan yang dibayar per jam sekaligus mendapatkan komisi dari total penjualan. Tantangannya adalah bagaimana merancang class baru yang mewarisi perilaku dari parent class namun juga menambahkan fungsionalitas baru tanpa mengubah class yang sudah ada. Untuk melengkapi studi kasus ini dibutuhkan beberapa hal: pertama, memahami hierarki class yang sudah ada (StaffMember → Employee → Hourly) agar Commission dapat ditempatkan dengan tepat dalam hierarki tersebut. Kedua, menambahkan dua instance variable baru yaitu totalSales dan commissionRate. Ketiga, meng-override method pay() dengan memanggil super.pay() untuk mendapatkan gaji jam kerja lalu menambahkan hasil perkalian totalSales * commissionRate sebagai komisi. Keempat, memperbarui Staff.java dengan menambah 2 objek Commission beserta data jam kerja dan penjualannya untuk menguji polimorfisme — di mana satu array StaffMember[] dapat menampung berbagai tipe karyawan berbeda dan memanggil pay() secara polimorfik.
Nama Teman Hal yang Dibantu (Opsional)
-

No. 2 Inheritance via Polymorphism (Painting Shapes)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 2-Polymorphism via Inheritance.pdf
Source Code
Paint.java

```java
//*******************************************************
// Paint.java
//
// Represents a type of paint that has a fixed area
// covered by a gallon. All measurements are in feet.
//*******************************************************
public class Paint {
    private double coverage; // number of square feet per gallon
    // ----------------------------------------
    // Constructor: Sets up the paint object.
    // ----------------------------------------

    public Paint(double c) {
        coverage = c;
    }

    // ------------------------------------------------
    // Returns the amount of paint (number of gallons)
    // needed to paint the shape given as the parameter.
    // ------------------------------------------------
    public double amount(Shape s) {
        System.out.println("Computing amount for " + s);
        return s.area() / coverage;
    }
}

```

Cylinder.java

```java
//*******************************************************************
// Cylinder.java
//
// Represents a cylinder shape.
//*******************************************************************
public class Cylinder extends Shape {
    private double radius;
    private double height;

    // -----------------------------------------------------------------
    // Constructor: Sets up the cylinder with given radius and height.
    // -----------------------------------------------------------------
    public Cylinder(double r, double h) {
        super("Cylinder");
        radius = r;
        height = h;
    }

    // -----------------------------------------------------------------
    // Returns the surface area of the cylinder (PI * radius^2 * height).
    // -----------------------------------------------------------------
    public double area() {
        return Math.PI * radius * radius * height;
    }

    // -----------------------------------------------------------------
    // Returns the cylinder as a String.
    // -----------------------------------------------------------------
    public String toString() {
        return super.toString() + " of radius " + radius + " and height " + height;
    }
}
```

PaintThings.java (main program)

```java

//*******************************************************************
// PaintThings.java
//
// Computes the amount of paint needed to paint various
// things. Uses the amount method of the paint class which
// takes any Shape as a parameter.
//*******************************************************************
import java.text.DecimalFormat;

public class PaintThings {
    // ----------------------------------------
    // Creates some shapes and a Paint object
    // and prints the amount of paint needed
    // to paint each shape.
    // ----------------------------------------
    public static void main(String[] args) {
        final double COVERAGE = 350;
        Paint paint = new Paint(COVERAGE);

        Rectangle deck;
        Sphere bigBall;
        Cylinder tank;
        double deckAmt, ballAmt, tankAmt;
        // Instantiate the three shapes to paint
        deck = new Rectangle(20, 35); // 20 x 35 foot rectangle
        bigBall = new Sphere(15); // sphere of radius 15
        tank = new Cylinder(10, 30); // cylinder radius 10, height 30
        // Compute the amount of paint needed for each shape
        deckAmt = paint.amount(deck);
        ballAmt = paint.amount(bigBall);
        tankAmt = paint.amount(tank);
        // Print the amount of paint for each.
        DecimalFormat fmt = new DecimalFormat("0.#");
        System.out.println("\nNumber of gallons of paint needed...");
        System.out.println("Deck " + fmt.format(deckAmt));
        System.out.println("Big Ball " + fmt.format(ballAmt));
        System.out.println("Tank " + fmt.format(tankAmt));
    }
}
```

Rectangle.java

```java
//*******************************************************************
// Rectangle.java
//
// Represents a rectangle shape.
//*******************************************************************
public class Rectangle extends Shape {
    private double length;
    private double width;

    // -----------------------------------------------------------------
    // Constructor: Sets up the rectangle with given length and width.
    // -----------------------------------------------------------------
    public Rectangle(double l, double w) {
        super("Rectangle");
        length = l;
        width = w;
    }

    // -----------------------------------------------------------------
    // Returns the area of the rectangle (length x width).
    // -----------------------------------------------------------------
    public double area() {
        return length * width;
    }

    // -----------------------------------------------------------------
    // Returns the rectangle as a String.
    // -----------------------------------------------------------------
    public String toString() {
        return super.toString() + " of length " + length + " and width " + width;
    }
}
```

Shape.java

```java
//*************************************************************************
// Shape.java
//
// Abstract class representing a generic shape.
//*************************************************************************
public abstract class Shape {
    private String shapeName;

    // -----------------------------------------------------------------
    // Constructor: Sets the name of the shape.
    // -----------------------------------------------------------------
    public Shape(String name) {
        shapeName = name;
    }

    // -----------------------------------------------------------------
    // Abstract method - each subclass must define how to compute area.
    // -----------------------------------------------------------------
    public abstract double area();

    // -----------------------------------------------------------------
    // Returns the name of the shape.
    // -----------------------------------------------------------------
    public String toString() {
        return shapeName;
    }
}
```

Sphere.java

```java
//*******************************************
// Sphere.java
//
// Represents a sphere.
//*******************************************
public class Sphere extends Shape {
    private double radius; // radius in feet
    // ----------------------------------
    // Constructor: Sets up the sphere.
    // ----------------------------------

    public Sphere(double r) {
        super("Sphere");
        radius = r;
    }

    // ----------------------------------------
    // Returns the surface area of the sphere.
    // ----------------------------------------
    public double area() {
        return 4 * Math.PI * radius * radius;
    }

    // ----------------------------------
    // Returns the sphere as a String.
    // ----------------------------------
    public String toString() {
        return super.toString() + " of radius " + radius;
    }
}

```

Screenshot Hasil

```
Computing amount for Rectangle of length 20.0 and width 35.0
Computing amount for Sphere of radius 15.0
Computing amount for Cylinder of radius 10.0 and height 30.0

Number of gallons of paint needed...
Deck 2
Big Ball 8.1
Tank 26.9
```

Penjelasan Permasalahan dan Solusi

Soal ini meminta pembuatan hierarki class bentuk geometri untuk menghitung kebutuhan cat. Tantangannya adalah membuat abstract class Shape sebagai parent, lalu menurunkannya ke Rectangle, Cylinder, dan Sphere, masing-masing dengan

rumus area yang berbeda. Selain itu, class Paint harus mampu menghitung kebutuhan cat untuk semua jenis Shape secara polimorfik. Untuk melengkapi studi kasus ini, pertama dibuat abstract class Shape dengan instance variable shapeName, abstract method area(), dan method toString(). Kedua, dibuat class Rectangle (area = length * width) dan Cylinder (area = PI * radius^2 * height) sebagai turunan Shape. Ketiga, method amount() pada Paint.java diperbaiki menjadi return s.area() / coverage agar menghitung galon cat secara polimorfik. Keempat, PaintThings.java dilengkapi dengan instansiasi tiga objek: deck (Rectangle 20x35), bigBall (Sphere radius 15), dan tank (Cylinder radius 10 tinggi 30), lalu memanggil paint.amount() untuk masing-masing.
Nama Teman Hal yang Dibantu (Opsional)
-

No. 3 Polymorphic Sorting

Soal Praktikum
Silahkan membaca soal pada file lampiran: 3-Polymorphic Sorting.pdf
Source Code
Numbers.java

```java

// *******************************************************
// Numbers.java
//
// Demonstrates insertionSort on an array of integers.
// *******************************************************
import java.util.Scanner;

public class Numbers {
    // ----------------------------------------
    // Reads in an array of integers, sorts them,
    // then prints them in sorted order.
    // ----------------------------------------
    public static void main(String[] args) {
        Integer[] intList; // asalnya int[] -> Integer[] agar bisa pakai Comparable
        int size;

        Scanner scan = new Scanner(System.in);
        System.out.print("\nHow many integers do you want to sort? ");
        size = scan.nextInt();

        intList = new Integer[size]; // perbaikan dari new int[] -> new Integer[]

        System.out.println("\nEnter the numbers...");
        for (int i = 0; i < size; i++)
            intList[i] = scan.nextInt();

        Sorting.insertionSort(intList); // diganti ke insertionSort (descending)

        System.out.println("\nYour numbers in sorted order (descending)...");
        for (int i = 0; i < size; i++)
            System.out.print(intList[i] + " ");
        System.out.println();
    }
}
```

Salesperson.java

```java
// ********************************************************
// Salesperson.java
//
// Represents a sales person who has a first name, last
// name, and total number of sales.
// ********************************************************
public class Salesperson implements Comparable {
    private String firstName, lastName;
    private int totalSales;

    // -------------------------------------------------------
    // Constructor: Sets up the sales person object with
    // the given data.
    // -------------------------------------------------------
    public Salesperson(String first, String last, int sales) {
        firstName = first;
        lastName = last;
        totalSales = sales;
    }

    // -----------------------------------------
    // Returns the sales person as a string.
    // -----------------------------------------
    public String toString() {
        return String.format("%-10s %-8s ]", lastName, firstName, totalSales);
    }

    // -----------------------------------------
    // Returns true if the sales people have
    // the same name.
    // -----------------------------------------
    public boolean equals(Object other) {
        return (lastName.equals(((Salesperson) other).getLastName()) &&
                firstName.equals(((Salesperson) other).getFirstName()));
    }

    // ------------------------------------------------
    // Order is based on total sales with the name
    // (last, then first) breaking a tie.
    // Negatif = this < other
    // Positif = this > other
    // ------------------------------------------------
    public int compareTo(Object other) {
        int result;
        Salesperson otherPerson = (Salesperson) other; // cast ke Salesperson

        // Bandingkan berdasarkan total sales
        result = this.totalSales - otherPerson.totalSales;

        // Jika sales sama (tie), gunakan nama belakang untuk memecah
        if (result == 0) {
            result = this.lastName.compareTo(otherPerson.lastName);

            // Jika nama belakang juga sama, gunakan nama depan
            if (result == 0)
                result = this.firstName.compareTo(otherPerson.firstName);
        }

        return result;
    }

    // ------------------------
    // First name accessor.
    // ------------------------
    public String getFirstName() {
        return firstName;
    }

    // ------------------------
    // Last name accessor.
    // ------------------------
    public String getLastName() {
        return lastName;
    }

    // ------------------------
    // Total sales accessor.
    // ------------------------
    public int getSales() {
        return totalSales;
    }
}
```

Sorting.java

```java
//********************************************************************
// Sorting.java Author: Lewis/Loftus
//
// Demonstrates the selection sort and insertion sort algorithms.
//********************************************************************
public class Sorting {
    // -----------------------------------------------------------------
    // Sorts the specified array of objects using the selection
    // sort algorithm. (ASCENDING order)
    // -----------------------------------------------------------------
    public static void selectionSort(Comparable[] list) {
        int min;
        Comparable temp;

        for (int index = 0; index < list.length - 1; index++) {
            min = index;
            for (int scan = index + 1; scan < list.length; scan++)
                if (list[scan].compareTo(list[min]) < 0)
                    min = scan;

            // Swap the values
            temp = list[min];
            list[min] = list[index];
            list[index] = temp;
        }
    }

    // -----------------------------------------------------------------
    // Sorts the specified array of objects using the insertion
    // sort algorithm. (DESCENDING order - diubah dari soal asli)
    // -----------------------------------------------------------------
    public static void insertionSort(Comparable[] list) {
        for (int index = 1; index < list.length; index++) {
            Comparable key = list[index];
            int position = index;

            // Shift SMALLER values to the right (untuk descending)
            // Perubahan: < 0 diganti > 0 agar urutan terbalik
            while (position > 0 && key.compareTo(list[position - 1]) > 0) {
                list[position] = list[position - 1];
                position--;
            }

            list[position] = key;
        }
    }
}
```

Strings.java

```java

// ********************************************************
// Strings.java
//
// Demonstrates insertionSort on an array of Strings.
// ********************************************************
import java.util.Scanner;

public class Strings {
    // ------------------------------------------
    // Reads in an array of strings, sorts them,
    // then prints them in sorted order.
    // ------------------------------------------
    public static void main(String[] args) {
        String[] strList;
        int size;

        Scanner scan = new Scanner(System.in);
        System.out.print("\nHow many strings do you want to sort? ");
        size = scan.nextInt();

        strList = new String[size];

        System.out.println("\nEnter the strings...");
        for (int i = 0; i < size; i++)
            strList[i] = scan.next();

        Sorting.insertionSort(strList); // insertionSort (descending)

        System.out.println("\nYour strings in sorted order (descending)...");
        for (int i = 0; i < size; i++)
            System.out.print(strList[i] + " ");
        System.out.println();
    }
}
```

WeeklySales.java

```java
// ****************************************************************
// WeeklySales.java
//
// Sorts the sales staff in descending order by sales.
// ****************************************************************
public class WeeklySales {
    public static void main(String[] args) {
        Salesperson[] salesStaff = new Salesperson[10];
        salesStaff[0] = new Salesperson("Raza", "Mulki", 3000);
        salesStaff[1] = new Salesperson("Haafiz", "Nayyara", 4935);
        salesStaff[2] = new Salesperson("Firon", "Fariz", 3000);
        salesStaff[3] = new Salesperson("Akmal", "Rizqillah", 2800);
        salesStaff[4] = new Salesperson("Nafidz", "Guntur", 1570);
        salesStaff[5] = new Salesperson("Nafis", "Cirebon", 3000);
        salesStaff[6] = new Salesperson("Ziddan", "Subang", 7300);
        salesStaff[7] = new Salesperson("Rama  ", "Kruw", 5000);
        salesStaff[8] = new Salesperson("Iqbal", "Cimahi", 2850);
        salesStaff[9] = new Salesperson("Kakang", "Kasep", 3000);

        Sorting.insertionSort(salesStaff); // descending

        System.out.println("\nRanking of Sales for the Week\n");
        for (Salesperson s : salesStaff)
            System.out.println(s);
    }
}
```

Screenshot Hasil

```
Ranking of Sales for the Week

Mulki       Raza       9000
Subang      Ziddan     7300
Kruw        Rama       5000
Nayyara     Haafiz     4935
Kasep       Kakang     3000
Fariz       Firon      3000
Cirebon     Nafis      3000
Cimahi      Iqbal      2850
Rizqillah   Akmal      2800
Guntur      Nafidz     1570
```

Penjelasan Permasalahan dan Solusi

1. Numbers.java menggunakan tipe primitif int[] yang tidak bisa digunakan dengan Comparable, sehingga menyebabkan error kompilasi karena Sorting hanya menerima Comparable[].
2. Method insertionSort pada Sorting.java hanya mengurutkan secara ascending (dari kecil ke besar), sedangkan soal meminta urutan descending (dari besar ke kecil).

Soal ini meminta penggunaan class Sorting yang mengimplementasikan selectionSort dan insertionSort untuk mengurutkan berbagai tipe objek yang mengimplementasikan interface Comparable. Tantangannya mencakup tiga hal: memperbaiki Numbers.java agar kompatibel dengan Comparable, mengubah insertionSort menjadi descending, dan melengkapi method compareTo pada Salesperson.java untuk mengurutkan berdasarkan total penjualan. Untuk melengkapi studi kasus ini, pertama tipe int[] pada Numbers.java diganti menjadi Integer[] karena Integer adalah wrapper class yang mengimplementasikan Comparable, sementara int (primitif) tidak bisa digunakan dengan Comparable[]. Kedua, kondisi pada insertionSort diubah dari < 0 menjadi > 0 agar menghasilkan urutan descending. Ketiga, method compareTo pada Salesperson diimplementasikan dengan membandingkan totalSales sebagai kriteria utama, lalu lastName dan firstName sebagai tiebreaker, sehingga WeeklySales menampilkan daftar tenaga penjual dari penjualan terbesar ke terkecil.
Nama Teman Hal yang Dibantu (Opsional)
-